BINUS INTERNATIONAL

Database Technology

Final Project Proposal

Supply Chain Management System

Submitted By:

Jovan Nikholas (2902641811)

Class L3AC

Computer Science Program

Table of Contents

1. Background

If you heard the term “Supply Chain Management” for the first time, you may think that this is a difficult topic. However, this system appears every day in our social life. For example if you go to a supermarket, have you ever thought that how they manage the product itself, the stock of a product, where do they got the product from, to whom they might sell the product. For this needs, a supply chain management is needed.

For company having a supply chain management system is a must. It is because by implementing this system they could efficiently manage their product flow, information, and resource while still meeting customer demands. By using SCM it integrates supplier, customer, product information, product stocks into a single application that every one in the company could use. The goal of using SCM is to ensure accurability such as ordering products from the right supplier, handling product stocks, inputting product information also having a way to track the data of the sales / purchase.

By using SCM it could help companies to reduce unnecessary cost and improving the company’s overall performance. By having accurate information such as product stocks, sales it prevents company to make a wrong purchase to the supplier. As a result, it allows the company to make better decision regarding purchasing and inventory management.

The other advantages of using SCM that it could improve communication between parties that we encounter when we buy / sell the products. For the “buying” part, suppliers, warehouse staff, manager, and sales team could access the same information from the system. In the other side the “selling part”, warehouse staff, manager, cashier could also access the information from the system. However the data that they access is different. For example cashier could not purchase product from the supplier, and purchasing can not do selling a product to the customer.

In conclusion, Supply Chain management system is an essential part at today’s business operation. It help companies manage their product, information and resource more efficiently and safely also while ensuring the customer’s need. By improving communication, reducing cost, using technology and increasing customer satisfaction, SCM contribute to the success and growth of a business.

2. Problem description

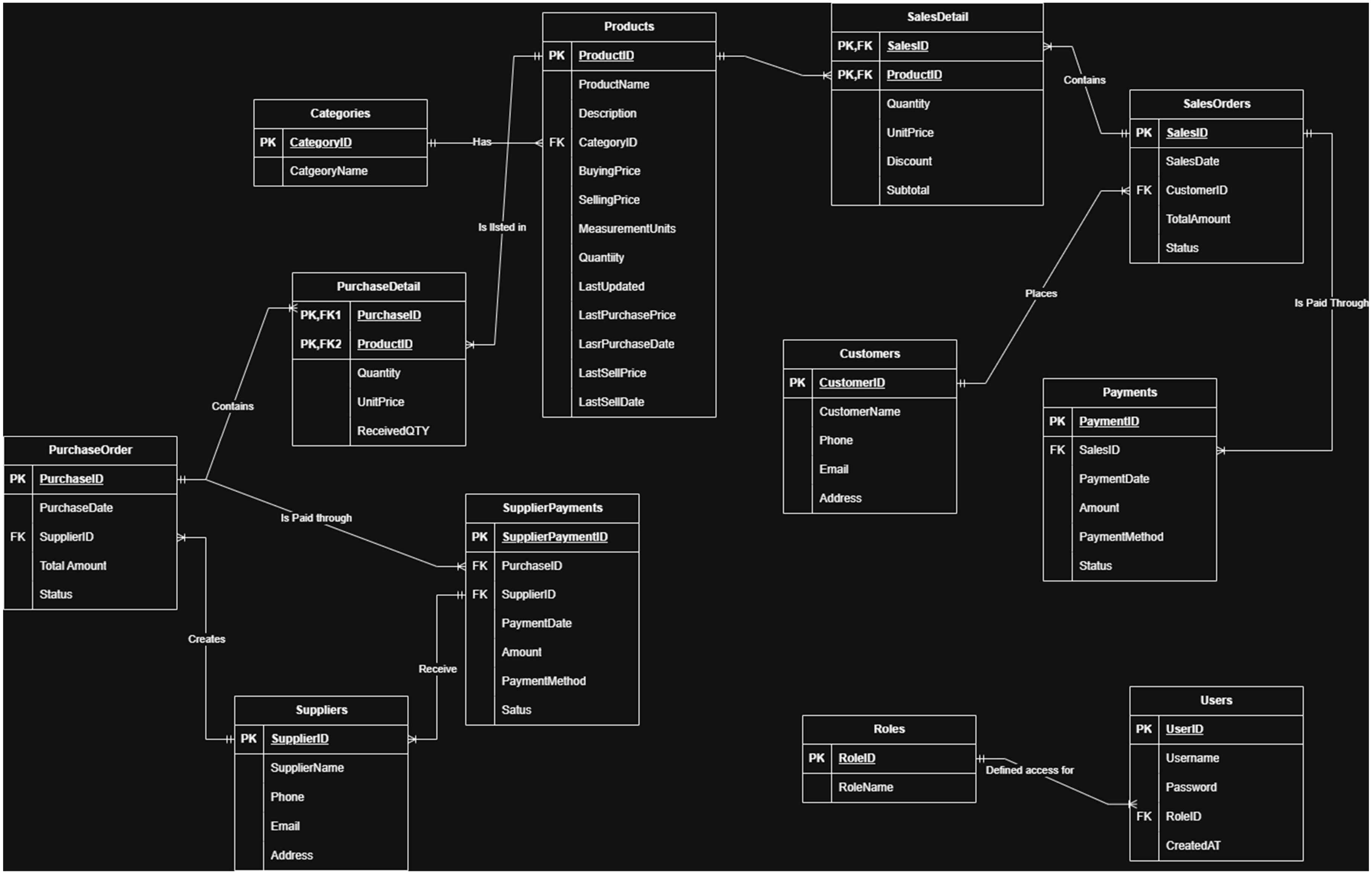
Despite the importance of Supply Chain Management System, many companies still have challenges in managing their business operation efficiently. In some companies some times product information, inventory record, supplier data, customer data, sales transaction, and purchase transaction are sometime is done manually or using separate system. By having it manually or in separate system could lead to inconsistencies, human error and difficulties in accessing information when it is needed.

The main problem of did not having SCM system is inventory management. Sometimes companies struggle to maintain their stock. For example having too much stocks could increase buying cost that is not necessary, while having too little stock could potentially reduce companies profit and lost sales opportunities.

The other problem is sometimes by not having SCM system is communication. Communication and coordination between different departments and supplier is sometimes could experience miscommunication. When an information is not shared efficiently, delay could occur in purchasing, production and product delivery. If the delay persist, it could affect the overall performance of the company and reduce customer satisfaction. Furthermore, tracking the movement of the product becomes more difficult when SCM system is not implemented.

Therefore, by having a proper Supply Chain Management system that could integrate supplier information, product data, inventory record, purchase transaction, sales transaction that integrates all the functionality into single platform could benefit the companies. It could benefit the company because it helps improve data accuracy, monitoring inventory level, enhance communication between workers.

3. ERD


Products
PK ProductID
ProductName
Description
FK CategoryID
BuyingPrice
SellingPrice
MeasurementUnits
Quantiity
LastUpdated
LastPurchasePrice
LasrPurchaseDate
LastSellPrice
LastSellDate
SalesDetail
PK,FK SalesID
PK,FK ProductID
Quantity
UnitPrice
Discount
Subtotal
Categories
PK CategoryID
CatgeoryName
Has
Is listed in
Contains
SalesOrders
PK SalesID
SalesDate
FK CustomerID
TotalAmount
Status
PurchaseDetail
PK,FK1 PurchaseID
PK,FK2 ProductID
Quantity
UnitPrice
ReceivedQTY
Places
Is Paid Through
Customers
PK CustomerID
CustomerName
Phone
Email
Address
Payments
PK PaymentID
FK SalesID
PaymentDate
Amount
PaymentMethod
Status
Contains
PurchaseOrder
PK PurchaseID
PurchaseDate
FK SupplierID
Total Amount
Status
Is Paid through
SupplierPayments
PK SupplierPaymentID
FK PurchaseID
FK SupplierID
PaymentDate
Amount
PaymentMethod
Satus
Creates
Receive
Suppliers
PK SupplierID
SupplierName
Phone
Email
Address
Roles
PK RoleID
RoleName
Defined access for
Users
PK UserID
Username
Password
FK RoleID
CreatedAT

In this application there is 12 table, and it is divided into 4 parts which are: Products, Customer sides, supplier sides, and user account sides. Products are used to define the products that we are going to sell. Then the product table is connected to categories. The relationship defined for both of this table is a product has 1 only 1 category, and for each category could be used to defined many products. Then we also have a table for user. User is going to be used for user access to use this application, each user has different access level for this webpage. For the users table it is connected to roles table. Roles table is used to define the roles of each user, to be used as a access control for the app. The connection for both of is table is a user could have only 1 role and for each roles defines access for a user.

For the supplier sides, it is started from the suppliers table. The Suppliers table is connected the PurchaseOrders table. The connection for this table is a supplier could create many purchase order, and only 1 purchase order belongs to 1 supplier. From there, the Purchaseorders table is linked with PurchaseDetail table. The connection for both of this table is one purchase order contain many purchase detail, and one purchase detail is only belong to 1 purchase order. Also there is a table for the supplier payment. This table is used to tell is a supplier has been paid by our company or not. This table is connected to with suppliers table. The relationship for this table is that supplier payment could be from many suppliers, and 1 supplier will have only 1 payment. Then the last one is the connection between PurchaseDetail table and product table. The connection for both of this table is product could belongs to many purchase detail, but purchase detail only belongs to one product.

For the Customer sides, it is started from the Customers table. The Customer table is connected to SalesOrder table. The relationship for this table is customer could have many sales order, but for each sales order is belongs to 1 customer. From there the SalesOrder table is connected to sales detail. The relationship for this table is a sales order could have many details, but a sales detail will belong to 1 sales order. Then the SalesDetail table is connected to products table. The connection for both this table is 1 sales detail is only belongs to 1 products and product could have many sales detail. The last connection is from SalesOrders table with Payments table. The connection for both this table is sales orders could have multiple payment but 1 payment only belongs to 1 sales orders.

4. Normalization

4.1 Unnormalized Form (UNF)

TransactioID	Transaction Type	Date	Name	Phone	Address	Products	Category	QTY	UnitPrice	Discount	Subtotal	SupplierName	SupplierPhone	Total Amount	Payment Method	Status
SO-001	Sales	25/5/2026	Bernice Divan	085777393609	Poris Paradise	Samsung Galaxy A57 , Samsung Galaxy J2 Prime	Phone, Phone	1,2	5000000 , 2500000	0, 0	5000000 , 5000000	Samsung , Samsung	081192988 , 081192988	10000000	Transfer	Paid
SO-002	Sales	25/5/2026	Nichola Barlianto	081929392004	Alam Sutera	Aqua 600ml , Tehbotol 600ml	Beverages , Beverages	5,5	5000 , 10000	0 , 0	25000 , 50000	Unilever	08161883452 , 08161883452	75000	Cash	Paid
PO-001	Purchase	20/5/2026	Samsung	081192988	Batam	Samsung Galaxy A57 , Samsung Galaxy J2 Prime	Phone , Phone	5,5	4000000 , 1500000	0,0	20000000 , 7500000	Samsung	081192988 , 081192988	27500000	Cheque	Paid
PO-002	Purchase	15/5/2026	Unilever	08161883452	DaanMogot	Aqua 600ml , Tehbotol 600ml	Beverages , Beverages	20,20	2500 , 5000	0 , 0	50000 , 100000	Unilever	08161883452	150000	Cheque	Paid

4.2 First Normal Form (1NF)

Sales Table

TransactionID	Transaction Type	Date	CustomerName	CustomerPhone	CustomerAddress	Products	QTY	UnitPrice	Discount	Subtotal	SupplierName	Supplier Phone	Total Amount	Payment Method	Status
SO-001	Sales	25/5/2026	Bernice Divan	085777393609	Poris Paradise	Samsung Galaxy A57	1	5000000	0	5000000	Samsung	081192988	10000000	Transfer	Paid
SO-001	Sales	25/5/2026	Bernice Divan	085777393609	Poris Paradise	Samsung Galaxy J2 Prime	2	2500000	0	5000000	Samsung	081192988	10000000	Transfer	Paid
SO-002	Sales	25/5/2026	Nichola Barlianto	081929392004	Alam Sutera	Aqua 600ml	5	5000	0	25000	Danone	08161883452	75000	Cash	Paid
SO-002	Sales	25/5/2026	Nichola Barlianto	081929392004	Alam Sutera	Tehbotol 600ml	5	10000	0	50000	Sosro	08367734831	75000	Cash	Paid

(PK,FK) :Transaction ID + Product Name

Purchase Tabel

TransactionID	Transaction Type	Date	SupplierName	SupplierPhone	SupplierAddress	ProductName	Category	Qty	UnitPrice	Subtotal	TotalAmount	PaymentMethod	Status
PO-001	Purchase	20/5/2026	Samsung	081192988	Batam	Samsung Galaxy A57	Phone	5	4000000	20000000	27500000	Cheque	Paid
PO-001	Purchase	20/5/2026	Samsung	081192988	Batam	Samsung Galaxy J2 Prime	Phone	5	1500000	7500000	27500000	Cheque	Paid
PO-002	Purchase	15/5/2026	Unilever	08161883452	DaanMogot	Aqua 600ml	Beverages	20	2500	50000	150000	Cheque	Paid
PO-002	Purchase	15/5/2026	Unilever	08161883452	DaanMogot	Tehbotol 600ml	Beverages	20	5000	100000	150000	Cheque	Paid

(PK,FK) : Transaction ID + Product Name

4.3 Second Normal Form (2NF)

SalesOrders : OrderID (PK)

OrderID	Date	CustomerName	CustomerPhone	CustomerAddress	TotalAmount	PaymentMethod	Status
SO-001	25/5/2026	Bernice Divan	085777393609	Poris Paradise	5000000	Transfer	Paid
SO-002	25/5/2026	Nichola Barlianto	081929392004	Alam Sutera	75000	Cash	Paid

PurchaseOrders : OrderID(PK)

OrderID	Date	SupplierName	SupplierPhone	SupplierAddress	Amount	PaymentMethod	Status
PO-001	20/5/2026	Samsung	081192988	Batam	27500000	Cheque	Paid
PO-002	15/5/2026	Unilever	08161883452	DaanMogot	150000	Cheque	Paid

Products : ProductName (PK)

ProductName	Category	SupplierName	SupplierPhone
Samsung Galaxy A57	Phone	Samsung	085777393609
Samsung Galaxy J2 Prime	Phone	Samsung	085777393609
Aqua 600ml	Beverages	Unilever	08161883452
Tehbotol 600ml	Beverages	Unilever	08161883452

SalesDetail Table : (PK,FK) : SalesOrders.OrderID (PK,FK) Products.ProductName

OrderID	ProductName	QTY	UnitPrice	Discount	Subtotal
SO-001	Samsung Galaxy A57	1	5000000	0	5000000
SO-001	Samsung Galaxy J2 Prime	2	2500000	0	5000000
SO-002	Aqua 600ml	5	5000	0	25000
SO-002	Tehbotol 600ml	5	10000	0	50000

PurchaseDetail Table (PK,FK) : PurchaseOrders.OrderID (PK,FK) Products.ProductName

OrderID	ProductName	QTY	UnitPrice	Discount	Subtotal
PO-001	Samsung Galaxy A57	5	4000000	0	20000000
PO-001	Samsung Galaxy J2 Prime	5	1500000	0	7500000
PO-002	Aqua 600ml	20	2500	0	50000
PO-002	Tehbotol 600ml	20	5000	0	100000

4.4 Third Normal Form (3NF)

CategoriesTable

CategoryID (PK)	CategoryNames
C001	Phone
C002	Beverages

CustomersTable:

CustomerID (PK)	CustomerName	CustomerPhone	CustomerAddress
CS001	Bernice Divan	085777393609	Poris Paradise
CS002	Nichola Barlianto	081929392004	Alam Sutera

SuppliersTable :

SupplierID (PK)	SupplierName	SupplierPhone	SupplierAddress
SP001	Samsung	081192988	Batam
SP002	Unilever	08161883452	DaanMogot

Products Table

ProductID (PK)	ProductName	CategoryID (FK) Categories.CategoryID	SupplierID (FK) Suppliers.SupplierID
P001	Samsung Galaxy A57	C001	SP001
P002	Samsung Galaxy J2 Prime	C001	SP001
P003	Aqua 600ml	C002	SP002
P004	Tehbotol 600ml	C002	SP002

SalesOrders Table :

OrderID (PK)	Date	CustomerID (FK) Customers.CustomersID	TotalAmount	PaymentMethod	Status
SO-001	25/5/2026	CS001	27500000	Transfer	Paid
SO-002	25/5/2026	CS002	150000	Cash	Paid

PurchaseOrders Table

PurchaseID (PK)	Date	SupplierID (FK) Suppliers.SupplierID	TotalAmount	PaymentMethod	Status
PO-001	20/5/2026	SP001	27500000	Cheque	Paid
PO-002	15/5/2026	SP002	150000	Cheque	Paid

SalesDetail Table :

OrderID (PK,FK) SalesOrders.OrderID	ProductID (PK,FK) Products.ProductID	Qty	UnitPrice	Discount	Subtotal
SO-001	P001	1	5000000	0	5000000
SO-001	P002	2	2500000	0	5000000
SO-002	P003	5	5000	0	25000
SO-002	P004	5	10000	0	50000

PurchaseDetail Table

OrderID (PK,FK) PurchaseOrders.OrderID	ProductID (PK,FK)Products.ProductID	Qty	UnitPrice	Discount	Subtotal
PO-001	P001	5	4000000	0	20000000
PO-001	P002	5	1500000	0	7500000
PO-002	P003	20	2500	0	50000
PO-002	P004	20	5000	0	100000

4.5 Explanation

For the Unnormalized form (UNF) the data is scattered around, in here the customer sides, supplier sides, and products is in one table. The example of a customer side are customer name, customer phone and address. For Supplier sides are supplier name, supplier phone and supplier address. For the Products sides is the actual product itself and categories. By doing this the table will looks messy. In Unnormalized Form multiple entries in one column is allowed. For example, looking at the products table, there is 2 products in single table “Samsung Galaxy A57, Samsung Galaxy J2 Prime”. By doing this reader of this table will find it difficult.

For the First normalized form (1NF) the table is divided in to 2 which are sales table and purchase table. For sales table it is divided into 16 parts which are TransactionID, TransactionType, Date, CustomerName, Customer Phone, CustomerAddress, Products, QTY, UnitPrice, Discount, Subtotal, SupplierName, SupplierPhone, TotalAmount, Payment Method, and status. For Purchase table it is divided into 14 parts which are TransactionID , TransactionType, Date, SupplierName, SupplierPhone, SupplierAddress, ProductName, Category, QTY, UnitPrice, SubTotal, TotalAmount, PaymentMethod, and status. In 1NF keys also being used. For this particular table we use composite key. For the both table the key are TransactionID + ProductName.

For the Second Normalized Form (2NF) we remove the particial dependency. Because of this 2NF table are divided into 5 table which are SalesOrders Table, PurchaseOrders Table, Products Table, SalesDetail table and PurchaseDetail table. For the SalesOrders the primary key (PK) is OrderID and the table it is divided into 8 attributes which are OrderID, Date, CustomerName, CustomerPhone, CustomerAddress, TotalAmount, PaymentMethod and status. For the PurchaseOrders table the Primary Key (PK) is OrderID, and the table is divided into 8 attributes which are OrderID, Date, CustomerName, CustomerPhone, CustomerAddress, TotalAmount, PaymentMethod and status. For the products table the PK is ProductName and it has 4 attributes which are ProductName, Category, SupplierName and SupplierPhone. For SalesDetail table we use two composite key (PK,FK) which are OrderID and ProductName, for the attribures there are 6 which are OrderID, ProductName, Quantity, UnitPrice, Discount and Subtotal. For PurchaseDetail table we use composite key (PK,FK) OrderID, ProductName. For this table there are in total 6 attributes which are OrderID, ProductName, Quantity, UnitPrice, Discount and Subtotal

For the Third Normalized Form (3NF) Transitive Dependency is removed. Because of that 3NF are divided into 8 table which are Categories Table, Customers Table, Suppliers Table, Products table, SalesOrders Table, PurchaseOrders Table, SalesDetail Table, and PurchaseDetail Table. For Categories Table it is divided into 2 attributes which are CategoryID (PK) and CategoryName. The Custoemrs Table are divided in to 4 attributes which are CustomerID (PK), CustomerName, Customer Phone and CustomerAddress. For Suppliers table it is alsoj divided into 4 attributes which are SupplierID(PK), SupplierName, SupplierPhone, and SupplierAddress. For products table it is divided into 4 attributes which are ProductID(PK), ProductName, CategoryID (FK), SupplierID (FK). For the SalesOrders Table it is divided into 6 attributes which are OrderID(PK), Date, CustomerID (FK), TotalAmount, PaymentMethod ans status. For PurhcaseOrders Table it is also divided into 6 attributes which are PurchaseID(PK), Date, SupplierID(FK), TotalAmount, PaymentMethod, and status. For SalesDetail table it is divided into 6 table which are OrderID, ProductID, QTY, UnitPrice, Discount, and subtotal. For this table we use composite key (PK,FK) which are OrderID and Product ID. For the PurchaseDetail Table we also divided it into 6 table which are OrderID, ProductID, QTY, UnitPrice, Discount, Subtotal

5. Table Structures

In this table we use 12 table which are :

1. Products Table

   For Products table we has 13 attributes which are ProductID with the data type INT, ProductName with the data type VARCHAR(45) , Description with the data type VARCHAR(45), CategoryID with the datatype VARCHAR(45), BuyingPrice with the datatype DECIMAL(10,2), SellingPrice with the data type DECIMAL(10,2), MeasurementUnits with the data type VARCHAR(45), Quantity with the data type INT, LastUpdated with the data type DATETIME, LastPurchasedPrice with the data type DECIMAL (10,2), LastPurchasedDate with the datatype DATETIME, LastSellPrice with the data type DECIMAL(10,2), LastSellDate with the datatype DATETIME. The Primary Key for this table is ProductID with the constrains that is Foreign KEY (FK) CategoryID which linked Categories Table CategoryID

2. Categories Table

   For Categories table we has 2 attributes which are CategoryID with the data type INT, and CategoryName with the data type VARCHAR(45). The Primary Key for this table is CategoryID.

3. PurchaseDetail Table

   For PurchaseDetail table we have 4 attributes which are PurchaseID with the data type INT, ProductID with the data type INT, Quantity with the data type INT, Unit Price with the data type Decimal (10,2), and ReceivedQty with the data type INT. For this we are using composite key (PK,FK) PurchaseID,ProductID. For the PurchaseID the key is from the PurchaseID inside PurchaseOrder Table, meanwhile the ProductID is the key from ProductID inside the table products,

4. PurchaseOrder Table

   For PurchaseOrder Table we have 4 attributes which are PurchaseID with the data type INT, PurchaseDate with the data type DATETIME, SupplierID with the data type INT, TotalAmount with the data type DECIMAL(10,2), and status with the data type VARCHAR(45). For this table the Primary Key(PK) is PurchaseID. The constrain for this table is the foreign key which are SupplierID which got the key from SupplierID in the table Suppliers.

5. Suppliers Table

   For Suppliers table we have 4 attributes which are SupplierID with the data type INT, SupplierName with the data type VARCHAR(45), Phone with the data type

VARCHAR (20), Email with the data type VARCHAR (45), Address with the data type VARCHAR(45). The primary key for this table is SupplierID

6. SupplierPayments Table
   For SupplierPayments Table we have 7 attributes which are SupplierPaymentsID with the data type INT, PurchaseID with the data type INT, Supplier ID with the data type INT, PaymentDate with the data type DATETIME, Amount with the data type DECIMAL(10,2), PaymentMethod with the datatype VARCHAR(45), Status With the data type VARCHAR(45). For Supplier Payments the PK is SupplierPaymentID. With the constrain of Foreign key PurchaseID and SupplierID. For PurchaseID it got the key from the table PurchaseOrder table with the the attributes PurchaseID. For SupplierID it got the key from the Suppleirs with the attributes SupplierID.

7. Customers Table
   For Customers Table there are 5 attributes which are CustomerID with the data type INT, CustomerName with the data type VARCHAR(45), Phone with the data type VARCHAR(45), Email with the data type VARCHAR(45), and Address with the data type VARHCAR(45). For the Customers Table the Primary Key is CustomerID

8. SalesOrders Table
   For SalesOrders table there are 5 attributes which are SalesID with the data type INT, SalesDate with the datatype DATETIME, CustomerID with the data type INT, TotalAmount with the datatype DECIMAL(10,2), and status with the data type VARCHAR(45). In this table the primary key is SalesID. The constrains is the Foreign key which is CustomerID with got the ID from Customers Table with the attributes CustomerID.

9. Payments Table
   For Payments Table there are 6 attributes which are PaymentID with the data type INT, SalesID with the data type INT, PaymentDate with the datatype DATETIME, Amount with the data type DECIMAL(10,2), PaymentMethod with the data type VARCHAR(45), Status with the data type VARCHAR(45). Fot this table the Primary Key is PaymentID. The Constrain of this table is Foreign key which is SalesID which got the key from SalesOrders table with the attributes SalesID.

10. SalesDetail Table

    For SalesDetail Table there are 6 attributes which are SalesID with the data type INT, ProductID with the data type INT, Quantity with the data type INT, UnitPrice with the data type DECIMAL(10,2), Discount with the data type Decimal(10,2), Subtotal with the datatype (10,2). This table uses composite key (PK,FK) SalesID, ProductID. With the Sales ID is got from the SalesOrders table attributes SalesID

11. Roles Table

    For Roles Table there are 2 attributes which are RoleID with the data type INT and RoleName with the data type VARCHAR(45). For the roles table the primary key is RoleID

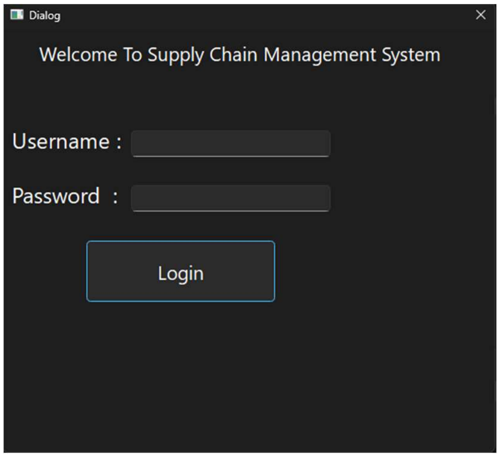
12. User Table

    For User Table there are 3 attributes which are UserID with the data type INT, Username with the data type VARCHAR(45), Password with the data type VARCHAR(255), RoleID with the data type INT, and CreatedAt with the data type INT. The primary key for this table is UserID. For the constrain is foreign key which are RoleID, the key is from Roles Table Attributes RoleID.

6. UI Design

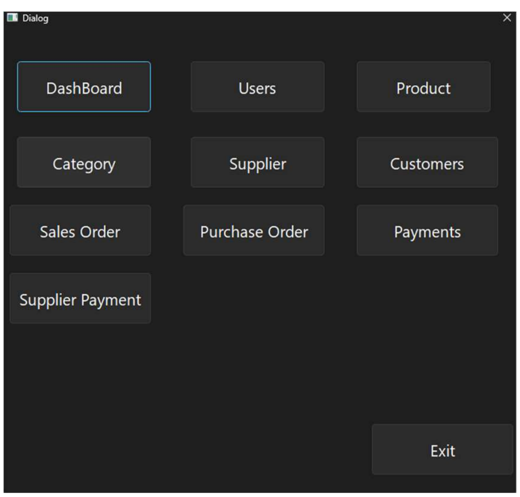
For the design of the UI, Python Library called Pyside6. In the pyside6 QT Widget Designer is used as a scene builder.

1. Login Screen

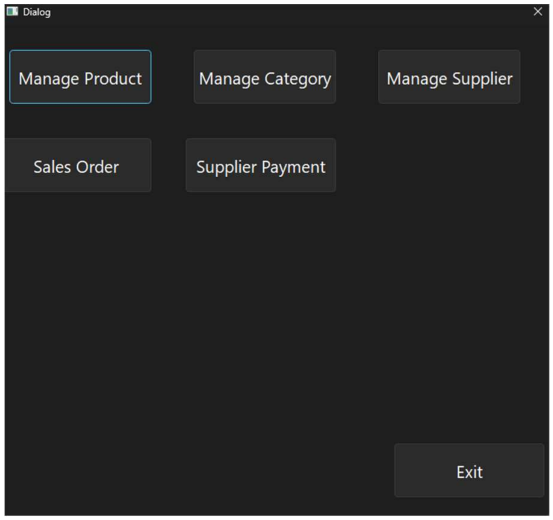


For the login screen, we have to input the username and password of the users. The Username and password will correspond to the dashboard features that it will gets. In this program an admin will got full capability of the app, the purchasing and the cashier will have limited functionality of the app.

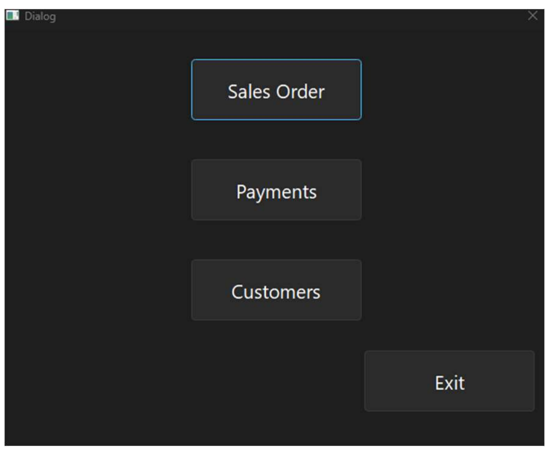
2. Main menu



This is the Dashboard if the user enters as admin credential. The admin could use all of the functionality that this app offers. It will have access to Dashboard, Users, Product, Category, Supplier, Customers, Sales Order, Purchase Order, Payments, and Supplier Payments

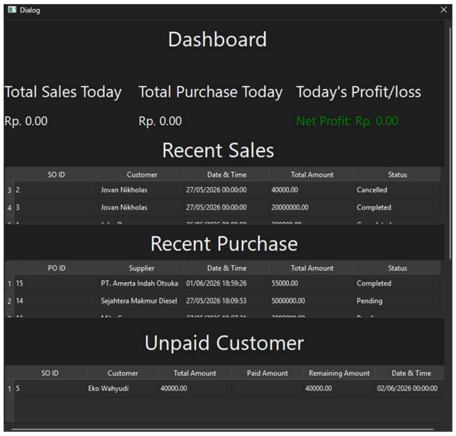


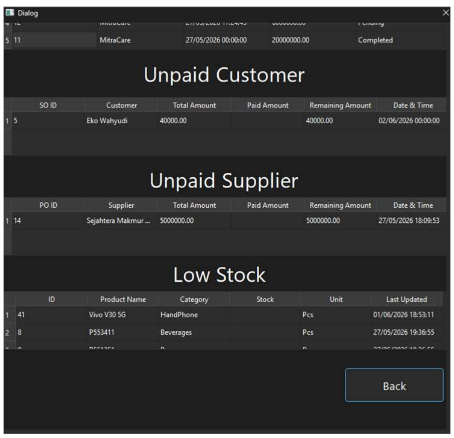
This is the example of the main menu if the users enter as purchasing credential, the main menu will only show the features that only the purchasing need. The features that could be used is Manage Product, Manage Category, Manage Supplier, Sales Order and Supplier Payment.



This is the example of main menu if the users enter as cashier prudential. The menu on the main menu that it will show is Sales Order, Payments and Customers.

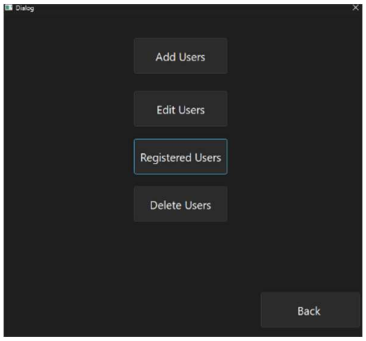
3. Dashboard





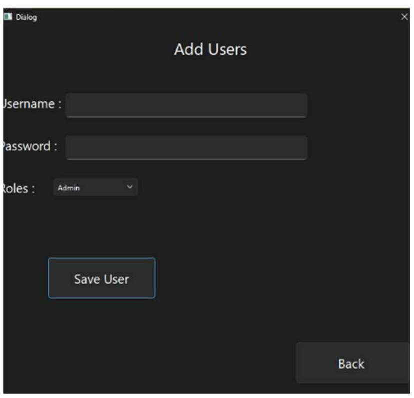
This is the UI design for the Dashboard menu. In The top we have Total Sales Today which are the sales that is made today, then we have total purchase today which are the total purchase that have been made from the supplier, also today profits/loss which calculate the total sales today – total purchase today. Below we have the Recent Sales which displays the last 5 sales that have been made. Below that we have Recent Purchase which displays the last 5 purchase that have been made from the supplier. Below that we have unpaid customers to display the customers that not yet paid. Below that we have unpaid supplier to show the suppliers that we have not paid. Below that is a low stock that display if the product stocks is less than 5.

4. User Menu



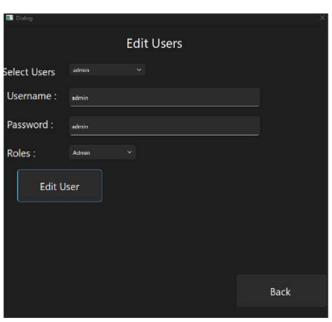
In user menu, thee are 4 options which are Add Users, Edit Users, Registered Users and Delete Users.

5. Add Users



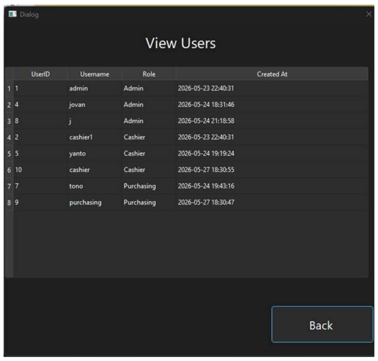
The function of add user menu is to add a new user in to the system. To add user we must specify the username, password and roles that is intended to put in the system. For the roles there are 3 roles which are admin, cashier, and purchasing

6. Edit User



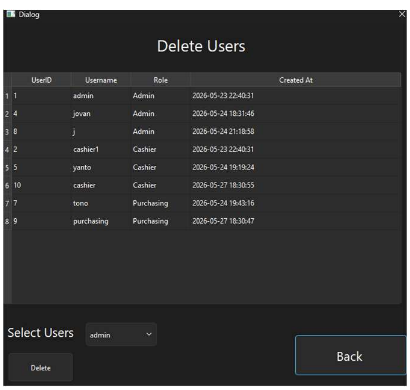
\The Edit User menu is used to edit user information to the sytem. First we have to select the users that we wanted to edit using the provided to combo box. After we select from the combo box the username, password and roles are filled automatically. From there we could modify the user credential, once finish click the edit user button and the edited user will be saved on the database.

7. View Users



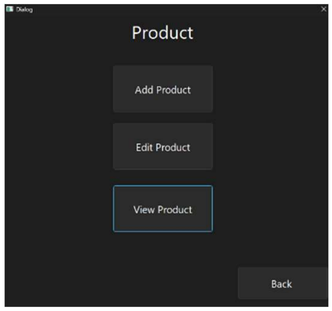
View users is used to view the registered user in the database. In here we could view the userid , Username, role and when is the user created.

8. Delete User



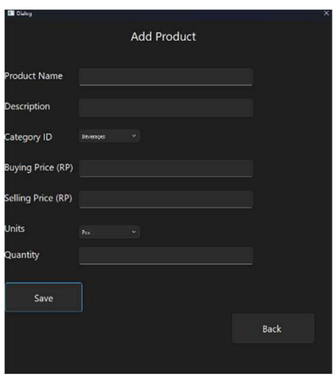
Delete User is used to delete user from the database. To delete the user click the provided combo box and choose the user that is going to be deleted. The last step is to click delete.

9. Product Menu



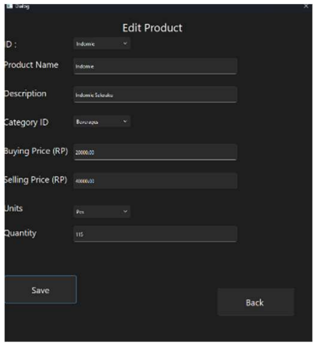
This is the menu for produce menu. It consists of 3 menu which are add product, edit product and view product. Add product is used to add new product to the system, edit product is used to edit the product that is already available in the sytem and view product is used to view registered product and to delete the product in the system.

10. Add Product



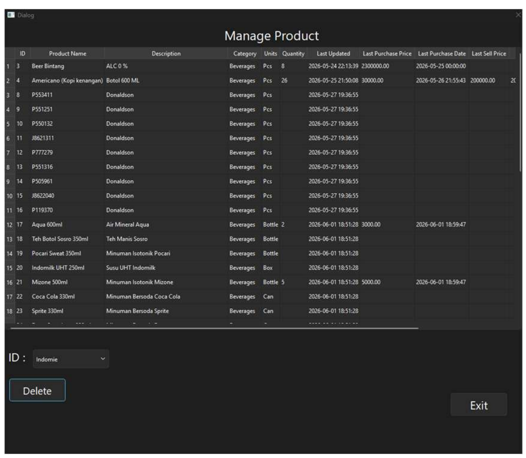
This is the menu to add new product to the system. To add a new product, user must enter product name, enter product description, choose a category that has already been set in category menu, enter the initial buying and selling price, enter product's unit and initial quantity. After all of that has been entered, click to save to add the product to the system

11. Edit Product



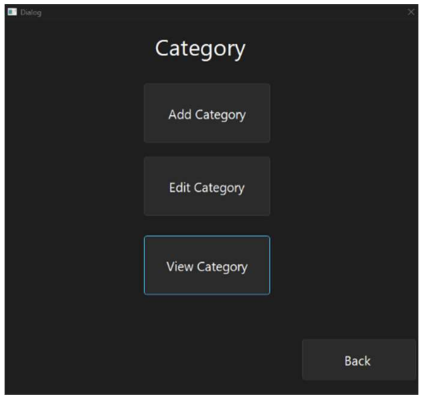
Edit Product is used to edit a product that has been registered in the system. To edit a product users has to use the combo box to select what product they are going to edit. After they choose the product that they wanted to edit, they could start edit the product name, description, category, buying and selling price, unit and quantity. After they edited, they click save to save the edited product.

12. Manage Product



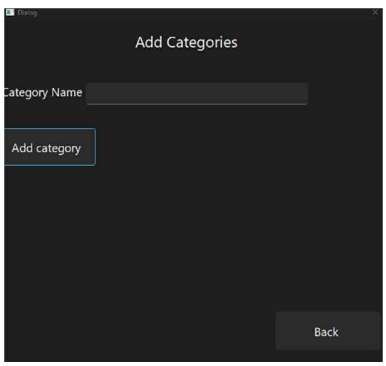
Manage Product is used to view and delete available product. To view the product user just scroll in the UI. In here they could see product ID, Product Name, Description, Category, Units, Quantity, Last Updated, Last Purchase Price, Last Sell Price and Last Sell Date. In the bottom part, user could choose what product they want to delete from the system.

13. Category Menu



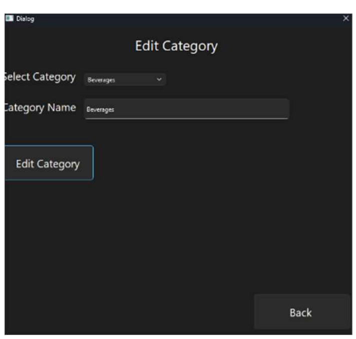
Category menu is used for the user to choose whether they want to add, edit and view category.

14. Add Category Menu



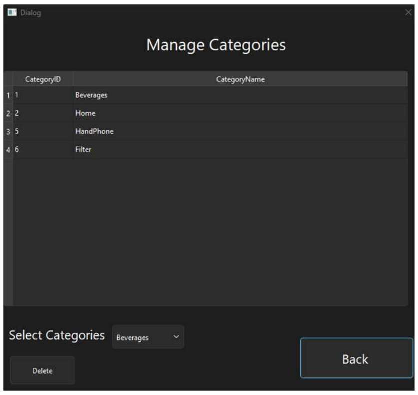
Add category menu, is used to add new category to the application. User of this program needs to enter category name, after they added they have to click the add category to save it to the database.

15. Edit Category Menu



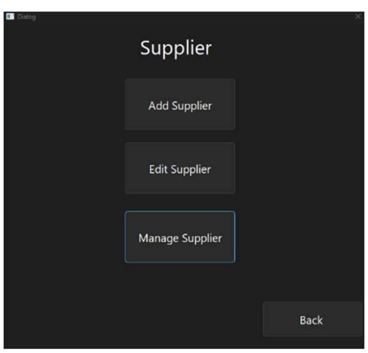
Edit Category menu is used to add edit the category that has been registered in the databases. To use it user need to choose the category that is available from the combo box. After they choose the category, they could change its name. When they finish they have to save it by clicking the edit category menu.

16. Manage Category



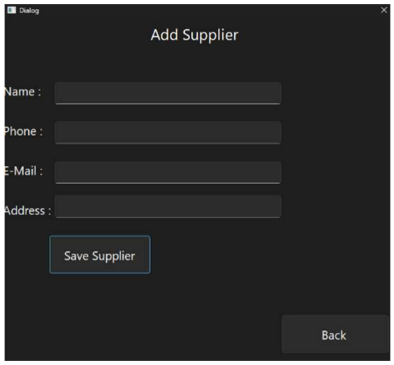
Manage Category is used to view which categories is already registered to the system. To view the category, user could scroll and this menu also serves to delete the categories, by using the combo box that is provided at the bottom of the application.

17. Supplier Menu



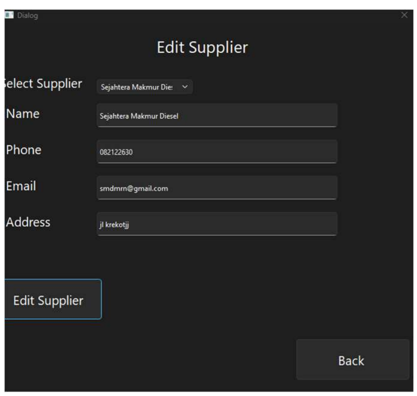
In the supplier menu, the user of this app could add new supplier, edit existing supplier, and they could view and delete the available supplier (Manage Supplier Button)

18. Add Supplier



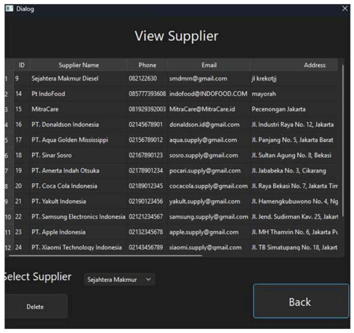
Add Supplier is used to add new supplier to the database. To add a new supplier to the database user needs to enter its name, phone, email and address. After they enter the information, to save to the system user have to click save.

19. Edit Supplier



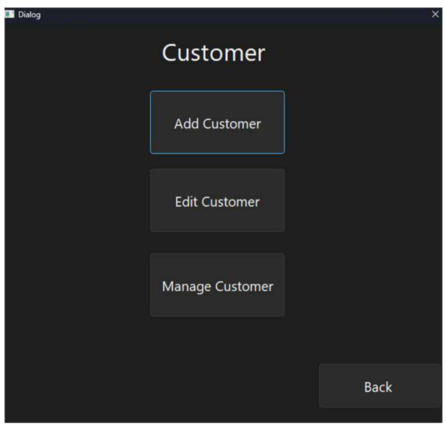
Edit supplier is used to edit the existing supplier and their information to the database. First user needs to select the registered supplier using the combo box, after they selected they could edit the supplier information. After they have finished they need to click “Edit Supplier” button to be saved to the system.

20. Manage Supplier



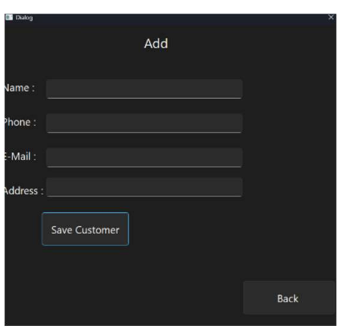
Manage Supplier is used to view supplier information and deleted it. User could view the supplier information such as its ID, Name, Phone, Email and address.

21. Customer Menu



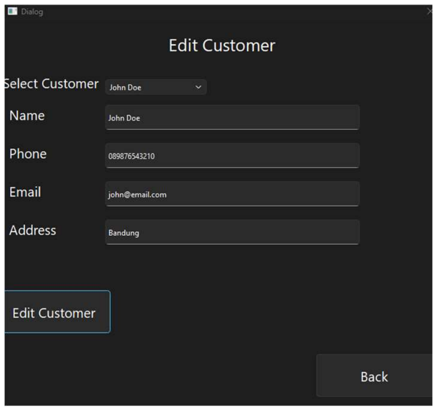
The function of customer menu is for the user to add, edit and manage new customer.

22. Add Customer



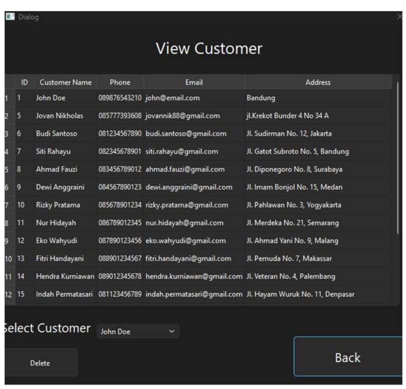
Add Customer is used to add new customer to the database, to add a new customer user has to input customer's name, phone, email and address. After they add the information, user has to click save customer.

23. Edit Customer



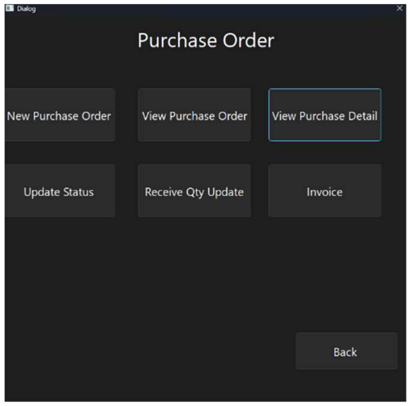
Edit customer is used to edit customer information. To edit he customer information, user has to select the customer information in the combo box. After they have selected, they could start editing the user information. Once, they have finished, they must click the edit customer button to be saved to the database.

24. Manage Customer



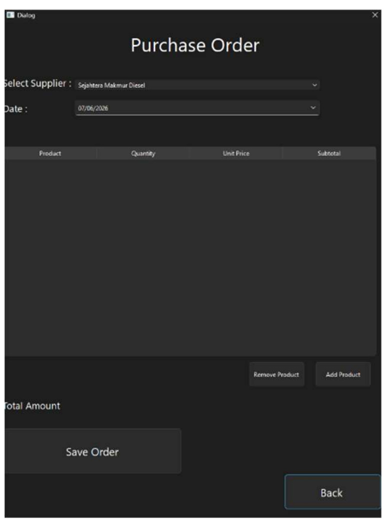
Manage Customer is used to view customer information such as its id, name, phone, email and address. In this menu users also could delete the customer from the database by clicking the combo box at the bottom of the screen and clicking the delete button.

25. Purchase Order Menu



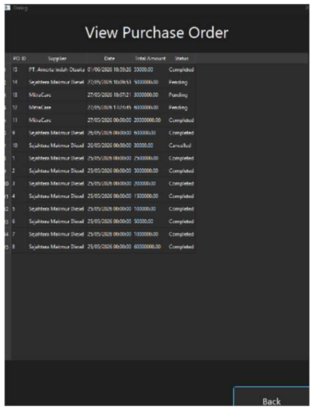
Purchase order menu is a menu to do everything that regards to purchase order to supplier such as creating a new purchase order, viewing purchase order, viewing purchase detail, update status, receive qty update and making an invoice for the supplier.

26. New Purchase Order



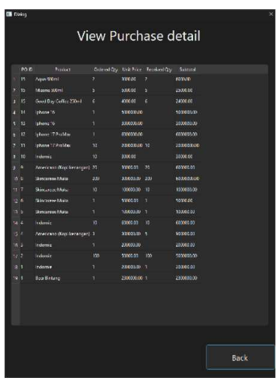
This menu is used to making a new purchase order for the supplier. To use this menu, we have to select the supplier from the list that has already been registered in to the system, after that we must specify the date of the purchase (the default day is today). From there to add the product to the form, click the "Add Product" button. From there we could choose the product that has already been registered in the system. After we choose the product we input the quantity, and the price. From there the calculation of the total will be done automatically. After finish input, click save order to save the purchase order.

27. View Purchase Order



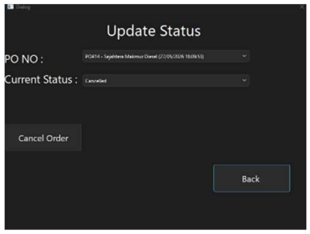
View purchase order is used to view the purchase order from the user that has already being made. In here we could see the PO ID, Supplier Name, Date, Total amount and status. The status will be set be default to pending if the received quantity has not been added (Go to the receive QTY menu)

28. View Purchase Detail



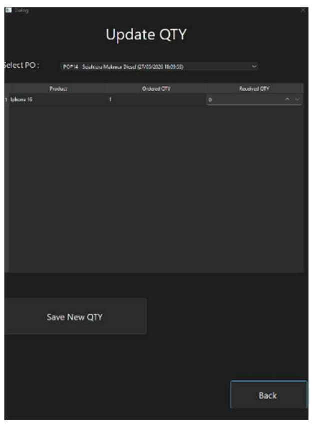
View purchase detail is used to view products that has been purchased through the supplier, viewing order quantity, the unit price, Received order quanity and the subtotal of the products.

29. Update Status



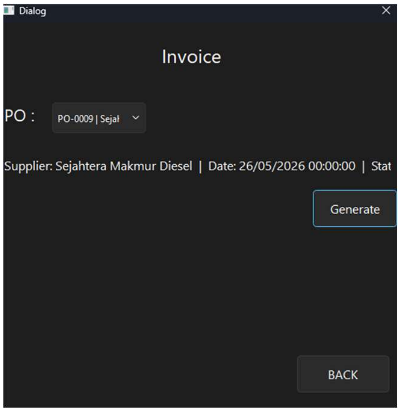
Update Status is used to cancel the purchase order that have been made. To use this feature we have to select the PO No using the combo box and click cancel order to cancel the Purchase Order.

30. Receive QTY Update

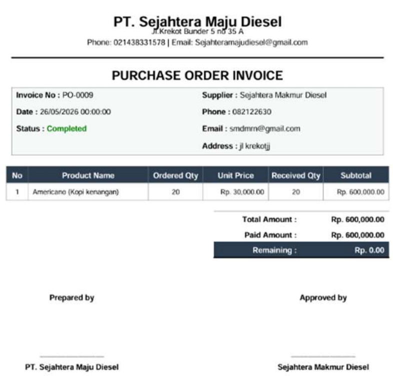


Receive QTY Update is used to update the quantity from the initial purchase order that has been made. To use this feature user have to select the Purchase Order using the combo box, and type the received qty. After that click the save new qty. If the quantity is already receive the status in view purchase order become completed.

31. Invoice (Purchase Order)



Invoice is used, to generate an invoice. To use this function user have to choose the purchase order that they wanted to generate the invoice. Then they click generate make the invoice. The result of the invoice is shown below.



PT. Sejahtera Maju Diesel
Jl.Krekot Bunder 5 no 35 A
Phone: 021438331578 | Email: Sejahteramajudiesel@gmail.com

PURCHASE ORDER INVOICE

Invoice No : PO-0009
Date : 26/05/2026 00:00:00
Status : Completed

Supplier : Sejahtera Makmur Diesel
Phone : 082122630
Email : smdmrn@gmail.com
Address : jl krekotjj

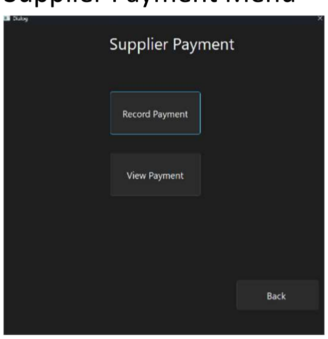
No	Product Name	Ordered Qty	Unit Price	Received Qty	Subtotal
1	Americano (Kopi kenangan)	20	Rp. 30,000.00	20	Rp. 600,000.00

Total Amount : Rp. 600,000.00
Paid Amount : Rp. 600,000.00
Remaining : Rp. 0.00

Prepared by
PT. Sejahtera Maju Diesel

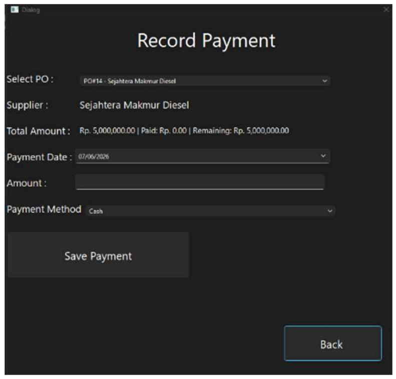
Approved by
Sejahtera Makmur Diesel

32. Supplier Payment Menu



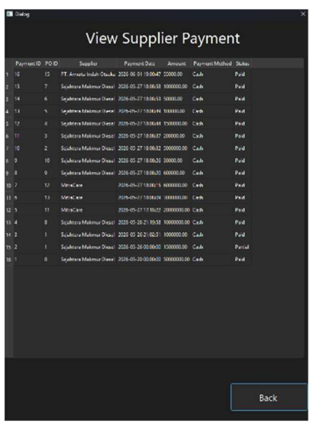
Supplier Payment is used to record supplier payment and view the payment status

33. Record Payment



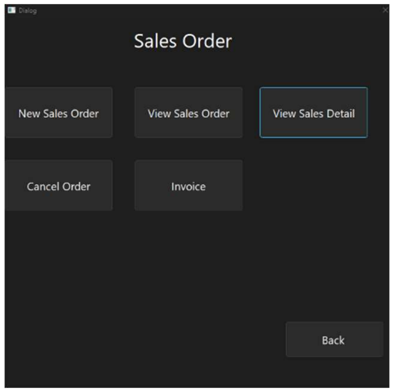
Record payment is used to record supplier payment, to use this feature we have to choose the purchase order from the combo box. Then the supplier name and total amount will be filled automatically. Then we have to choose the payment date (default date is today). Then we have to type it the amount and choose the payment method.

34. View Supplier Payment



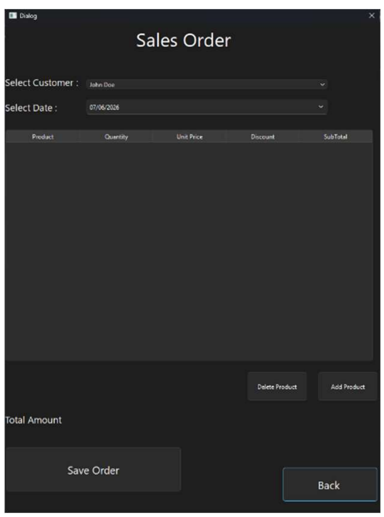
View supplier payment is used to tell the payment status of the purchase order. If the order has been paid it will show paid, if the payment is not full it will show as partial.

35. Sales Order Menu



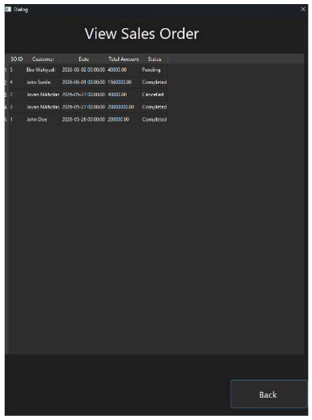
Sales Order menu is used to show all available features regarding sales order such as making new sales order, viewing sales order, viewing sales detail, cancelling order and making an invoice.

36. New Sales Order



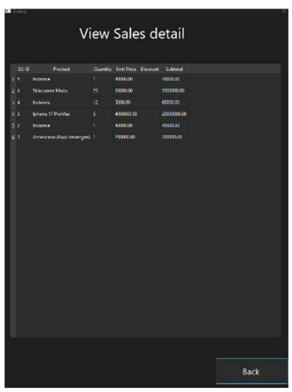
New sales order is used to make a new sales order for the customer. To use this feature we have to choose from the registered customer in the database. From there we have to select the date (The default date is today). Then we have to add the product that we wanted to sell by click the “Add Product” button. From there we find the product that we wanted, type the quantity, unit price, discount. The total will be done automatically by the system.

37. View Sales Order



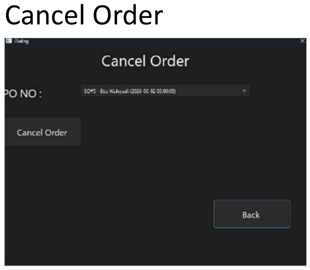
View Sales Order is used to see if the Sales order status. In here we could see the sales order id, customer, order date and status (the default status is pending).

38. View Sales Detail



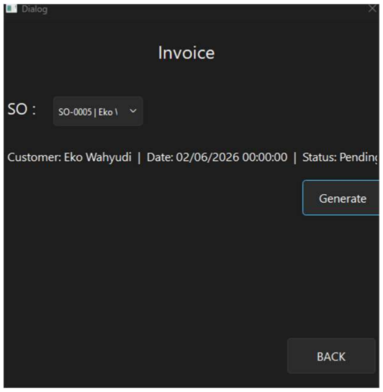
View Sales detail is used to view products that has been sold. In here we could see the Sales Order ID, Product Name, Quantity, Unit price, Discount and the subtotal.

39. Cancel Order

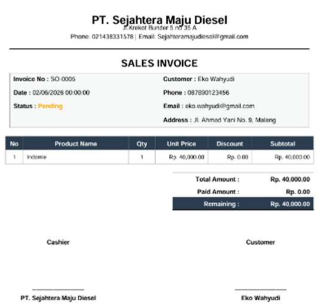


Cancel order is used to cancel the sales order that have been made.

40. Invoice (Sales Order)



This is used to create invoice for the sales order. To use this feature we have to select the invoice that we wanted to print using the combo box. From there we have to click generate to create the invoice. The invoice result will be shown below.



PT. Sejahtera Maju Diesel
Jl. Krekot Bunder 5 no 35 A
Phone: 021438331578 | Email: Sejahteramajudiesel@gmail.com

SALES INVOICE

Invoice No : SO-0005
Date : 02/06/2026 00:00:00
Status : Pending

Customer : Eko Wahyudi
Phone : 087890123456
Email : eko.wahyudi@gmail.com
Address : Jl. Ahmad Yani No. 9, Malang

No	Product Name	Qty	Unit Price	Discount	Subtotal
1	Indomie	1	Rp. 40,000.00	Rp. 0.00	Rp. 40,000.00

Total Amount : Rp. 40,000.00
Paid Amount : Rp. 0.00
Remaining : Rp. 40,000.00

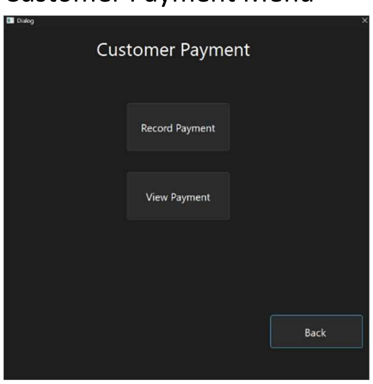
Cashier

PT. Sejahtera Maju Diesel

Customer

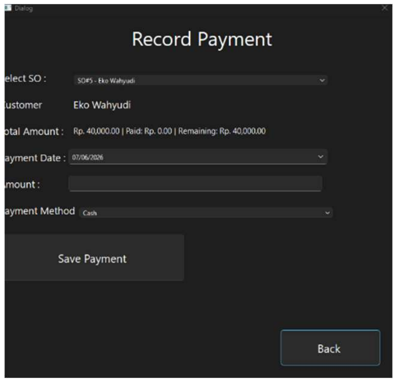
Eko Wahyudi

41. Customer Payment Menu



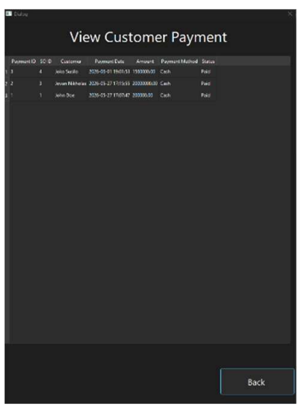
Customer Payment menu is used to record the payment for the customer also to view the payment from the customer.

42. Record payment



For the record payment it is used to record a payment for the customers. To do this we must select the sales order that is previously being made in the system. Then the customer name, and the total amount will be filled automatically. The users of this program have to choose the payment date, amount and the payment method. After that we click the save payment to save the payment.

43. View Customer Payment



The view customer payment is used to view the payment has already made or not.

7. SQL Code Appendix

7.1 Data Definition Language (DDL)

1. Creating the Database :
```
CREATE SCHEMA IF NOT EXISTS `scm` DEFAULT CHARACTER SET
utf8mb3; USE `scm`;
```

2. Creating Table Categories :
```
CREATE TABLE IF NOT EXISTS `scm`.`categories` (
  `CategoryID` INT NOT NULL AUTO_INCREMENT,
  `CategoryName` VARCHAR(45) NULL DEFAULT NULL,
  PRIMARY KEY (`CategoryID`))
ENGINE = InnoDB
AUTO_INCREMENT = 2
DEFAULT CHARACTER SET = utf8mb3;
```

3. Creating Table Suppliers
```
CREATE TABLE IF NOT EXISTS `scm`.`Suppliers` (
  `SupplierID` INT NOT NULL AUTO_INCREMENT,
  `SupplierName` VARCHAR(45) NULL,
  `Phone` VARCHAR(20) NULL,
  `Email` VARCHAR(45) NULL,
  `Address` VARCHAR(45) NULL,
  PRIMARY KEY (`SupplierID`))
ENGINE = InnoDB;
```

4. Creating Table PurchaseOrders
```
CREATE TABLE IF NOT EXISTS `scm`.`PurchaseOrders` (
  `PurchaseID` INT NOT NULL AUTO_INCREMENT,
  `PurchaseDate` DATETIME NULL,
  `SupplierID` INT NULL,
  `TotalAmount` DECIMAL(10,2) NULL,
  `Status` VARCHAR(45) NULL,
  PRIMARY KEY (`PurchaseID`),
  INDEX `SupplierID_idx` (`SupplierID` ASC) VISIBLE,
  CONSTRAINT `fk_purchaseorders_suppliers`
    FOREIGN KEY (`SupplierID`)
    REFERENCES `SCM`.`Suppliers` (`SupplierID`)
    ON DELETE NO ACTION
```

```
  ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

5. Creating Table Products

```
CREATE TABLE IF NOT EXISTS `scm`.`Products` (
 `ProductID` INT NOT NULL AUTO_INCREMENT,
 `ProductName` VARCHAR(45) NULL,
 `Description` VARCHAR(45) NULL,
 `CategoryID` INT NULL,
 `BuyingPrice` DECIMAL(10,2) NULL,
 `SellingPrice` DECIMAL(10,2) NULL,
 `MeasurementUnits` VARCHAR(45) NULL,
 `Quantity` INT NULL,
 `LastUpdated` DATETIME NULL,
 `LastPurchasePrice` DECIMAL(10,2) NULL,
 `LastPurchaseDate` DATETIME NULL,
 `LastSellPrice` DECIMAL(10,2) NULL,
 `LastSellDate` DATETIME NULL,
 PRIMARY KEY (`ProductID`),
 INDEX `CategoryID_idx` (`CategoryID` ASC) VISIBLE,
 CONSTRAINT `fk_products_categories`
  FOREIGN KEY (`CategoryID`)
  REFERENCES `SCM`.`Categories` (`CategoryID`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

6. Creating Table PurchaseDetail

```
CREATE TABLE IF NOT EXISTS `scm`.`PurchaseDetail` (
 `PurchaseID` INT NOT NULL,
 `ProductID` INT NOT NULL,
 `Quantity` INT NULL,
 `UnitPrice` DECIMAL(10,2) NULL,
 `ReceivedQty` INT NULL,
 PRIMARY KEY (`PurchaseID`, `ProductID`),
 INDEX `ProductID_idx` (`ProductID` ASC) VISIBLE,
 CONSTRAINT `fk_purchasedetail_purchaseorders`
  FOREIGN KEY (`PurchaseID`)
  REFERENCES `SCM`.`PurchaseOrders` (`PurchaseID`)
  ON DELETE NO ACTION
```

```
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_purchasedetail_products`
    FOREIGN KEY (`ProductID`)
    REFERENCES `SCM`.`Products` (`ProductID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

7. Creating Table Customers

```
CREATE TABLE IF NOT EXISTS `scm`.`Customers` (
  `CustomerID` INT NOT NULL AUTO_INCREMENT,
  `CustomerName` VARCHAR(45) NULL,
  `Phone` VARCHAR(45) NULL,
  `Email` VARCHAR(45) NULL,
  `Address` VARCHAR(45) NULL,
  PRIMARY KEY (`CustomerID`))
ENGINE = InnoDB;
```

8. Creating Table SalesOrders

```
CREATE TABLE IF NOT EXISTS `scm`.`SalesOrders` (
  `SalesID` INT NOT NULL AUTO_INCREMENT,
  `SalesDate` DATETIME NULL,
  `CustomerID` INT NULL,
  `TotalAmount` DECIMAL(10,2) NULL,
  `Status` VARCHAR(45) NULL,
  PRIMARY KEY (`SalesID`),
  INDEX `CustomerID_idx` (`CustomerID` ASC) VISIBLE,
  CONSTRAINT `fk_salesorders_customers`
    FOREIGN KEY (`CustomerID`)
    REFERENCES `SCM`.`Customers` (`CustomerID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

9. Creating Table SalesDetail

```
CREATE TABLE IF NOT EXISTS `scm`.`SalesDetail` (
  `SalesID` INT NOT NULL,
  `ProductID` INT NOT NULL,
  `Quantity` INT NULL,
  `UnitPrice` DECIMAL(10,2) NULL,
  `Discount` DECIMAL(10,2) NULL,
```

```
  `Subtotal` DECIMAL(10,2) NULL,
  PRIMARY KEY (`SalesID`, `ProductID`),
  INDEX `ProductID_idx` (`ProductID` ASC) VISIBLE,
  CONSTRAINT `fk_salesdetail_salesorders`
    FOREIGN KEY (`SalesID`)
    REFERENCES `SCM`.`SalesOrders` (`SalesID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_salesdetail_products`
    FOREIGN KEY (`ProductID`)
    REFERENCES `SCM`.`Products` (`ProductID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

10. Creating Table Payments

```
CREATE TABLE IF NOT EXISTS `scm`.`Payments` (
  `PaymentID` INT NOT NULL AUTO_INCREMENT,
  `SalesID` INT NULL,
  `PaymentDate` DATETIME NULL,
  `Amount` DECIMAL(10,2) NULL,
  `PaymentMethod` VARCHAR(45) NULL,
  `Status` VARCHAR(45) NULL,
  PRIMARY KEY (`PaymentID`),
  INDEX `SalesID_idx` (`SalesID` ASC) VISIBLE,
  CONSTRAINT `fk_payments_salesorders`
    FOREIGN KEY (`SalesID`)
    REFERENCES `SCM`.`SalesOrders` (`SalesID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

11. Creating Table SupplierPayments

```
CREATE TABLE IF NOT EXISTS `scm`.`SupplierPayments` (
  `SupplierPaymentsID` INT NOT NULL AUTO_INCREMENT,
  `PurchaseID` INT NULL,
  `SupplierID` INT NULL,
  `PaymentDate` DATETIME NULL,
  `Amount` DECIMAL(10,2) NULL,
  `PaymentMethod` VARCHAR(45) NULL,
  `Status` VARCHAR(45) NULL,
```

```
  PRIMARY KEY (`SupplierPaymentsID`),
  INDEX `PurchaseID_idx` (`PurchaseID` ASC) VISIBLE,
  CONSTRAINT `fk_supplierpayments_purchaseorders`
    FOREIGN KEY (`PurchaseID`)
    REFERENCES `SCM`.`PurchaseOrders` (`PurchaseID`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

12. Creating User Table

```
CREATE TABLE IF NOT EXISTS `scm`.`users` (
  `UserID` INT NOT NULL AUTO_INCREMENT,
  `Username` VARCHAR(45) NOT NULL,
  `Password` VARCHAR(255) NOT NULL,
  `RoleID` INT NULL DEFAULT NULL,
  `CreatedAt` DATETIME NULL DEFAULT NULL,
  PRIMARY KEY (`UserID`),
  INDEX `fk_users_roles` (`RoleID` ASC) VISIBLE,
  CONSTRAINT `fk_users_roles`
    FOREIGN KEY (`RoleID`)
    REFERENCES `scm`.`roles` (`RoleID`))
ENGINE = InnoDB
AUTO_INCREMENT = 4
DEFAULT CHARACTER SET = utf8mb3;
```

13. Creating Roles Table

```
CREATE TABLE IF NOT EXISTS `scm`.`roles` (
  `RoleID` INT NOT NULL AUTO_INCREMENT,
  `RoleName` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`RoleID`))
ENGINE = InnoDB
AUTO_INCREMENT = 4
DEFAULT CHARACTER SET = utf8mb3;
```

7.2 Data Manipulation Languages (DML)

1. Use The Database
   use scm;

2. Insert Roles example :
   INSERT INTO Roles (RoleName) VALUES ('Admin');

3. Insert Users Example :
   INSERT INTO Users (Username, Password, RoleID, CreatedAt)
   VALUES ('admin', 'admin123', 1, NOW());

4. Insert Category Example :
   INSERT INTO Categories (CategoryName) VALUES ('Filter');

5. Insert Supplier Example :
   INSERT INTO Suppliers (SupplierName, Phone, Email, Address)
   VALUES
   ('PT. Donaldson Indonesia', '02145678901',
   'donaldson.id@gmail.com', 'Jl. Industri Raya No. 12, Jakarta');

6. Insert Product Example :
   INSERT INTO Products (ProductName, Description, CategoryID,
   BuyingPrice, SellingPrice, MeasurementUnits, Quantity,
   LastUpdated) VALUES ('Aqua 600ml', 'Air Mineral Aqua', 2, 0, 0,
   'Bottle', 0, NOW());

7. Insert Customer Example :
   INSERT INTO Customers (CustomerName, Phone, Email, Address)
   VALUES
   ('Budi Santoso', '081234567890', 'budi.santoso@gmail.com', 'Jl.
   Sudirman No. 12, Jakarta');

8. Update Purchase Order Status (Auto update from system)
   UPDATE PurchaseOrders SET Status = 'Completed' WHERE
   PurchaseID = %s;
   Status could be = ‘pending’ , ‘completed’ , ‘cancelled’

9. Update Sales Order Status (Auto update from system)
   UPDATE SalesOrders SET Status = 'Completed' WHERE SalesID = %s;
   Status could be = ‘completed’ , ‘cancelled’

10. Update product qty (Auto update from system) (
    For adding Product (Buying)
    UPDATE Products SET Quantity = Quantity + %s,
      LastPurchasePrice = %s,
      LastPurchaseDate = NOW()
    WHERE ProductID = %s;
    For minus product (Selling)
    UPDATE Products SET Quantity = Quantity - %s,
      LastSellPrice = %s,
      LastSellDate = NOW()
    WHERE ProductID = %s;

11. Update Received QTY :
    UPDATE PurchaseDetail SET ReceivedQty = %s
    WHERE PurchaseID = %s AND ProductID = %s;

12. Delete (User,Product,Categories,Customer,Suppliers)
    DELETE FROM Users WHERE UserID = %s;
    DELETE FROM Products WHERE ProductID = %s;
    DELETE FROM Categories WHERE CategoryID = %s;
    DELETE FROM Customers WHERE CustomerID = %s;
    DELETE FROM Suppliers WHERE SupplierID = %s;

7.3 Queries

1. Login Authentication :
```
SELECT Users.UserID, Users.Username, Roles.RoleName
FROM Users
JOIN Roles ON Users.RoleID = Roles.RoleID
WHERE Users.Username = %s AND Users.Password = %s;
```

2. Get All Categories :
```
SELECT CategoryID, CategoryName FROM Categories;
```

3. Get All Suppliers :
```
SELECT SupplierID, SupplierName FROM Suppliers;
```

4. Get All Products :
```
SELECT ProductID, ProductName FROM Products;
```

5. Get Product for product info :
```
SELECT p.ProductID, p.ProductName, p.Description,
    c.CategoryName, p.MeasurementUnits, p.Quantity,
    DATE_FORMAT(p.LastUpdated, '%d/%m/%Y %H:%i:%s'),
    p.LastPurchasePrice,
    DATE_FORMAT(p.LastPurchaseDate, '%d/%m/%Y %H:%i:%s'),
    p.LastSellPrice,
    DATE_FORMAT(p.LastSellDate, '%d/%m/%Y %H:%i:%s')
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID;
```

6. Get All customers :
```
SELECT CustomerID, CustomerName FROM Customers;
```

7. Get all user id with roles (user info):
```
SELECT Users.UserID, Users.Username, Roles.RoleName,
Users.CreatedAt FROM Users JOIN Roles ON Users.RoleID =
Roles.RoleID;
```

8. Get All Purchase Orders :

```
SELECT po.PurchaseID, s.SupplierName,
    DATE_FORMAT(po.PurchaseDate, '%d/%m/%Y %H:%i:%s'),
    po.TotalAmount, po.Status
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
ORDER BY po.PurchaseDate DESC;
```

9. Get Purchase Detail :

```
SELECT pd.PurchaseID, p.ProductName, pd.Quantity,
    pd.UnitPrice, pd.ReceivedQty,
    (pd.Quantity * pd.UnitPrice) as Subtotal
FROM PurchaseDetail pd
JOIN Products p ON pd.ProductID = p.ProductID
JOIN PurchaseOrders po ON pd.PurchaseID = po.PurchaseID
ORDER BY pd.PurchaseID DESC;
```

10. Get Pending Purchase Orders:

```
SELECT po.PurchaseID, s.SupplierName,
    DATE_FORMAT(po.PurchaseDate, '%d/%m/%Y %H:%i:%s')
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
WHERE po.Status = 'Pending'
ORDER BY po.PurchaseDate DESC;
```

11. Get All Sales Orders :

```
SELECT so.SalesID, c.CustomerName,
    DATE_FORMAT(so.SalesDate, '%d/%m/%Y %H:%i:%s'),
    so.TotalAmount, so.Status
FROM SalesOrders so
JOIN Customers c ON so.CustomerID = c.CustomerID
ORDER BY so.SalesDate DESC;
```

12. Get Sales Detail :

```
SELECT sd.SalesID, p.ProductName, sd.Quantity,
    sd.UnitPrice, sd.Discount, sd.Subtotal
FROM SalesDetail sd
JOIN Products p ON sd.ProductID = p.ProductID
ORDER BY sd.SalesID DESC;
```

13. Get Unpaid Sales Orders :

```
SELECT so.SalesID, c.CustomerName, so.TotalAmount,
    COALESCE(SUM(p.Amount), 0) as TotalPaid
FROM SalesOrders so
JOIN Customers c ON so.CustomerID = c.CustomerID
LEFT JOIN Payments p ON so.SalesID = p.SalesID
WHERE so.Status != 'Cancelled'
GROUP BY so.SalesID, c.CustomerName, so.TotalAmount
HAVING COALESCE(SUM(p.Amount), 0) < so.TotalAmount
ORDER BY so.SalesDate DESC;
```

14. Get Unpaid Purchase Orders :

```
SELECT po.PurchaseID, s.SupplierName, po.TotalAmount,
    COALESCE(SUM(sp.Amount), 0) as TotalPaid
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
LEFT JOIN SupplierPayments sp ON po.PurchaseID = sp.PurchaseID
GROUP BY po.PurchaseID, s.SupplierName, po.TotalAmount
HAVING COALESCE(SUM(sp.Amount), 0) < po.TotalAmount
ORDER BY po.PurchaseDate DESC;
```

15. Get All Payment :

```
SELECT p.PaymentID, so.SalesID, c.CustomerName,
    DATE_FORMAT(p.PaymentDate, '%d/%m/%Y %H:%i:%s'),
    p.Amount, p.PaymentMethod, p.Status
FROM Payments p
JOIN SalesOrders so ON p.SalesID = so.SalesID
JOIN Customers c ON so.CustomerID = c.CustomerID
ORDER BY p.PaymentDate DESC;
```

16. Get All Supplier Payments :

```
SELECT sp.SupplierPaymentsID, po.PurchaseID, s.SupplierName,
    DATE_FORMAT(sp.PaymentDate, '%d/%m/%Y %H:%i:%s'),
    sp.Amount, sp.PaymentMethod, sp.Status
FROM SupplierPayments sp
JOIN PurchaseOrders po ON sp.PurchaseID = po.PurchaseID
JOIN Suppliers s ON sp.SupplierID = s.SupplierID
ORDER BY sp.PaymentDate DESC;
```

17. Total Sales Today (Dashboard)

```
SELECT COALESCE(SUM(Amount), 0)
FROM Payments
WHERE DATE(PaymentDate) = CURDATE()
```

18. Total Purchase Today (Dashboard)

```
SELECT COALESCE(SUM(Amount), 0)
FROM SupplierPayments
WHERE DATE(PaymentDate) = CURDATE();
```

19. Gross Profit/Loss (Dashboard)

```
SELECT
  COALESCE(SUM(sd.Quantity * sd.UnitPrice - sd.Discount), 0) -
  COALESCE(SUM(sd.Quantity * p.BuyingPrice), 0)
FROM SalesDetail sd
JOIN Products p ON sd.ProductID = p.ProductID
JOIN SalesOrders so ON sd.SalesID = so.SalesID
WHERE DATE(so.SalesDate) = CURDATE()
AND so.Status != 'Cancelled';
```

20. Last 5 Sales (Dashboard)

```
SELECT so.SalesID, c.CustomerName,
    DATE_FORMAT(so.SalesDate, '%d/%m/%Y %H:%i:%s'),
    so.TotalAmount, so.Status
FROM SalesOrders so
JOIN Customers c ON so.CustomerID = c.CustomerID
ORDER BY so.SalesDate DESC
LIMIT 5;
```

21. Last 5 Purchase (Dashboard)

```
SELECT po.PurchaseID, s.SupplierName,
    DATE_FORMAT(po.PurchaseDate, '%d/%m/%Y %H:%i:%s'),
    po.TotalAmount, po.Status
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
ORDER BY po.PurchaseDate DESC
LIMIT 5;
```

22. Unpaid Customers (Dashboard)

```
SELECT so.SalesID, c.CustomerName,
    so.TotalAmount,
    COALESCE(SUM(p.Amount), 0) as PaidAmount,
    so.TotalAmount - COALESCE(SUM(p.Amount), 0) as
RemainingAmount,
    DATE_FORMAT(so.SalesDate, '%d/%m/%Y %H:%i:%s')
FROM SalesOrders so
JOIN Customers c ON so.CustomerID = c.CustomerID
LEFT JOIN Payments p ON so.SalesID = p.SalesID
WHERE so.Status != 'Cancelled'
GROUP BY so.SalesID, c.CustomerName, so.TotalAmount,
so.SalesDate
HAVING COALESCE(SUM(p.Amount), 0) < so.TotalAmount
ORDER BY RemainingAmount DESC;
```

23. Unpaid Suppliers (Dashboard)

```
SELECT po.PurchaseID, s.SupplierName,
    po.TotalAmount,
    COALESCE(SUM(sp.Amount), 0) as PaidAmount,
    po.TotalAmount - COALESCE(SUM(sp.Amount), 0) as
RemainingAmount,
    DATE_FORMAT(po.PurchaseDate, '%d/%m/%Y %H:%i:%s')
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
LEFT JOIN SupplierPayments sp ON po.PurchaseID = sp.PurchaseID
WHERE po.Status != 'Cancelled'
GROUP BY po.PurchaseID, s.SupplierName, po.TotalAmount,
po.PurchaseDate
HAVING COALESCE(SUM(sp.Amount), 0) < po.TotalAmount
ORDER BY RemainingAmount DESC;
```

24. Low Stock Products (Dashboard)

```
SELECT p.ProductID, p.ProductName, c.CategoryName,
    p.Quantity, p.MeasurementUnits,
    DATE_FORMAT(p.LastUpdated, '%d/%m/%Y %H:%i:%s')
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.Quantity < 10
ORDER BY p.Quantity ASC;
```

25. Check Duplicate username (in users)

```
SELECT UserID FROM Users WHERE Username = %s;
```

26. Check If PO already Paid :

```
SELECT po.PurchaseID, s.SupplierName, po.TotalAmount,
    COALESCE(SUM(sp.Amount), 0) as TotalPaid
FROM PurchaseOrders po
JOIN Suppliers s ON po.SupplierID = s.SupplierID
LEFT JOIN SupplierPayments sp ON po.PurchaseID = sp.PurchaseID
GROUP BY po.PurchaseID, s.SupplierName, po.TotalAmount
HAVING COALESCE(SUM(sp.Amount), 0) < po.TotalAmount
ORDER BY po.PurchaseDate DESC;
```

27. Check stock before selling :

```
SELECT Quantity, ProductName FROM Products WHERE ProductID
= %s;
```

28. Get purchase detail (Update Receive QTY)

```
SELECT pd.ProductID, p.ProductName, pd.Quantity, pd.ReceivedQty
FROM PurchaseDetail pd
JOIN Products p ON pd.ProductID = p.ProductID
WHERE pd.PurchaseID = %s;
```

29. For Checking all items received

```
SELECT COUNT(*) FROM PurchaseDetail
WHERE PurchaseID = %s AND ReceivedQty < Quantity;
```

8. Conclusion

When I first choose the topic Supply Chain Management as my topic for this project to be honest I am feeling a little bit intimidated. It is because I know what is Supply Chain Management are, and I already know from the ground up it is going to be hard. In the supply chain management there will be 3 main component such as buying side, selling side and the product itself. I have to think how to integrate this main component into single application.

Before I started the final project, I have to present the proposal to our lecturer. From there I draw the ERD and gave my ERD to the lecturer. The lecturer gave me critics from my ERD. I take that critics to improve my ERD. The lecturer said that my ERD looks incomplete because I have not implemented two way relationship, also in the first ERD I do not have table for payment, the lecturer said it is best to add payment table. The other critics that I got is my ERD is messy, at that time I do not know how to use Draw.io properly so the connection is not happening automatically. From there I listen the critics that lecturer gave me and improve my ERD even more.

The other things that I learned from this project is right now I could design a database from scratch. I think this project challenge me to use everything that I learn from the class and learning new things from the internet. Because of this project also, I know that python has scene builder just like JavaFX. Before that if I wanted to make an UI using python I have to use ktinker with no built in scene builder.

The other challenge that I faced was a teamwork issue. Originally this project is two person project. However, due to unseen circumstances I made this project by my own. I take this as a challenge for myself, and as a practice to do everything individually. It is hard at first, but for sure it could be done.

9. Appendix

Github Link : https://github.com/jovannik88/DB_FinalProject_SCM_L3AC